

Basic information about the submission

Codebase: [Personal GitHub repository](#)

HW: Windows 11 + WSL2

Distribution: Ubuntu-24.04

Docker: 28.4.0

Languages used: Python (3.12.7) and JavaScript

Instructions

The implementation can be set up with the following commands:

```
git clone -b exercise1 git@github.com:Maitoproteiini/devops\_course\_fall\_2025.git
cd devops_course_fall_2025
docker compose up --build -d
curl localhost:8199/status
curl localhost:8199/log
cat ./vstorage/log.txt
```

To clean the persistent storages, use the bash file `clean_persistent_storages.sh`

```
sudo bash ./clean_persistent_storages.sh
```

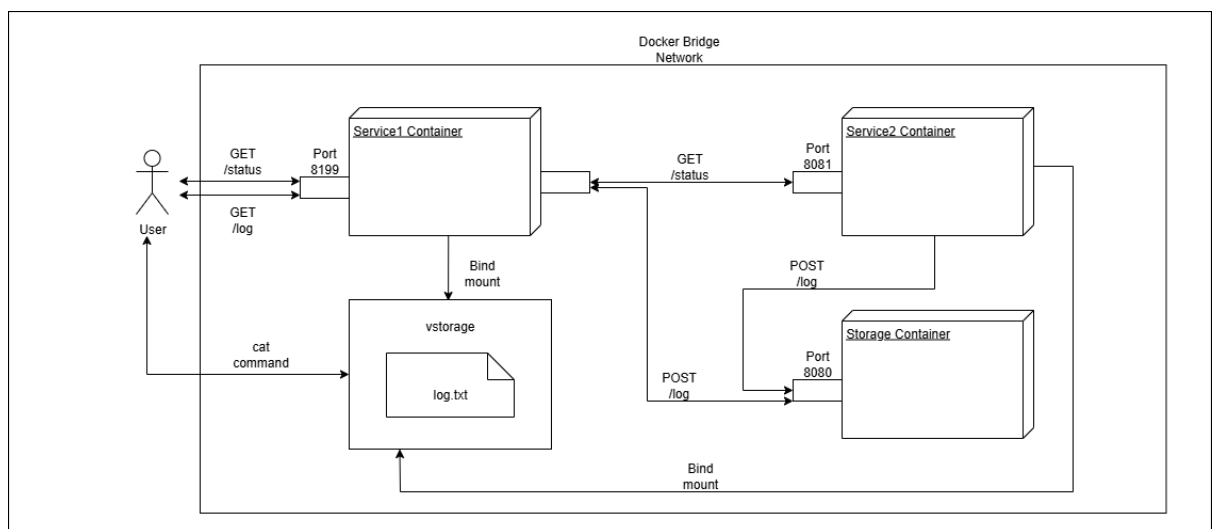
Introduction:

Three services were created. Two written in python called service1 and storage and the third written in JavaScript called service2. All three services run in separate docker containers.

Service1 acts as forwarding port to storage and service2. The user can request the status records of service1 and service2 by calling the `/status` endpoint served by service1. In addition, the user may retrieve all recorded status logs from storage by calling the `/log` endpoint served by service1.

The storage service stores all status logs requested from service1 and service2 persistently. This means that if the storage container is reset, it still holds all the logs once the container is back up again.

For educational purposes an additional storage was created called vstorage. This storage is not a service, but a bind-mounted host directory. This directory then holds log files created when users use the other services. The following diagram was drawn of the architecture.



Status Record Analysis

Uptime was read from /proc/uptime file. This file's first field gives the number of seconds since the kernel booted. The uptime was then returned as hours. This number is not the same as the time the container or the service has been up. A container may restart without the uptime resetting.

For service1 the amount of free disk space was retrieved with the python's shutil module. For service2 the number was retrieved with the "df -k" -command. Both methods indicate how much free disk space is available inside the container root.

These methods are relevant when containers are failing. One can quickly check if a container is full and temporary files can't be written. In addition, kernel uptime is a good metric when the network is hosted on a VM. A developer can quickly just request a GET request to check if the VM is up without attempting to connect to the VM via ssh connection.

However, it might be more useful to measure the actual storage used, since that is the focus of the service network. In addition, service uptime might be more useful to verify if the service has restarted recently. Finally, it might be more interesting to know how much disk space is available and not just free. Those values might not always be the same and available space is practically more relevant to a developer.

Storage Comparison

Two persistent storage solutions were implemented. Generally, it is thought that bind mounting a host directory is bad design. One reason is that containers should not have access to the hosts files. A bug in the code could be exploited to manipulate the host machine by overwriting the vstorage files.

Another reason is that it makes sharing the codebase more difficult. All users who want to copy the code and implement it on their end will need to modify the code so that file access and creation in the code matches the way it needs to be done on their host system. In addition, all the permissions and paths must be double checked. The directory layout must match the original hosts layout.

The design has some upsides to it as well. One, it's much simpler and quicker to implement. A developer does not need to create another service and make sure all other services have access to the container that the storage service is running in. The design is great, if a developer just needs to demo a proof of concept in a meeting.

The storage service implementation is however generally thought to be better design. For one, it's more portable. The same portability issues don't occur here. Second, it is more secure. A developer is not giving the services access to any host machine files. Instead, there is a clear boundary between the services and the host machine.

Third, because only the storage service writes to files, a developer does not have to worry about any race conditions. It's possible in the vstorage implementation that two services will attempt to write to the shared file at the same time.

There are a few downsides. It is more complicated to install. It also does make the whole system more complex and debugging may take more time. In addition, using a storage service does add latency. It will take more time to store data.

Final Thoughts

Using docker and setting up the services were not difficult. However, testing and building the codebase so that it could be just set up with minimal effort was. All projects tend to spend most of their time in development, so deployment is an area I have not spent as much time on. The main problems I encountered were testing and making sure all the requirements had been met.